

Scalable Data Processing

Batch Data Processing with PySpark

Niteesh K R

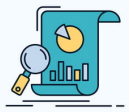
Computer Applications

Scalable Data Processing

Batch Data Processing with PySpark

Spark – Overview, Architecture, RDDs

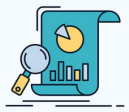
Niteesh K R
Computer Applications



Scalable Data Processing

Apache Spark – Introduction

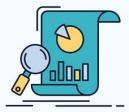
- Apache Spark is a distributed computing framework designed for large-scale data processing
- It provides fast, in-memory computation capabilities
- Spark supports batch processing, interactive queries, and stream processing
  - Designed to overcome limitations of MapReduce
  - Optimised for iterative and low-latency workloads



Scalable Data Processing

Why Apache Spark Was Introduced

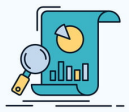
- Traditional MapReduce relies heavily on disk I/O between stages
- This results in high latency and inefficient iterative processing
- Spark addresses these issues through in-memory computation
  - Reduced disk access
  - Faster job execution
  - Improved developer productivity



Scalable Data Processing

Key Features of Apache Spark

- Spark offers a rich set of features for distributed analytics
  - In-memory processing
  - Fault tolerance through lineage
  - Unified analytics engine
  - Support for multiple programming languages
- Spark applications can scale from a laptop to large clusters

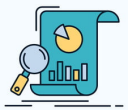


Scalable Data Processing

Apache Spark – High-Level Architecture

PES UNIVERSITY

- Spark follows a master–worker architecture
- A Spark application consists of a driver program and executors
  - Driver coordinates the execution
  - Executors perform computation and store data
- Cluster manager allocates resources for Spark applications

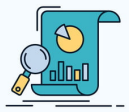


Scalable Data Processing

Components of Spark Architecture

PES UNIVERSITY

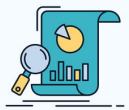
- Major components involved in Spark execution
  - Driver Program
  - Cluster Manager
  - Worker Nodes
  - Executors
- Each component plays a distinct role in distributed execution



Scalable Data Processing

Driver Program – Responsibilities

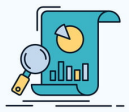
- The driver program is the central control point of a Spark application
- It runs the user's main function
  - Creates SparkContext
  - Builds execution plan
  - Coordinates tasks and collects results
- Failure of the driver terminates the application



Scalable Data Processing

Cluster Manager

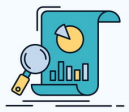
- Cluster manager is responsible for resource allocation
- Spark supports multiple cluster managers
  - Standalone
  - YARN
  - Kubernetes
- Spark remains independent of the underlying cluster manager



Scalable Data Processing

Executors and Worker Nodes

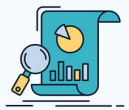
- Executors run tasks and store data in memory or disk
- They are launched on worker nodes
  - Execute tasks assigned by driver
  - Cache intermediate data
  - Report status back to driver



Scalable Data Processing

Spark Application Execution Flow

- Spark application execution follows a defined flow
  - Driver submits application
  - Cluster manager allocates resources
  - Executors execute tasks
- Execution is optimised through parallelism and data locality

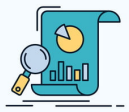


Scalable Data Processing

SparkContext – Introduction

PES UNIVERSITY

- SparkContext is the entry point to the Spark execution environment
- It represents a connection to the Spark cluster
- Only one SparkContext is active per application
  - Coordinates distributed operations
  - Manages cluster resources

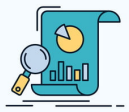


Scalable Data Processing

Creating SparkContext in PySpark

PES UNIVERSITY

- In PySpark, SparkContext is typically created implicitly
- SparkSession internally manages SparkContext
  - `from pyspark.sql import SparkSession`
  - `spark = SparkSession.builder.getOrCreate()`
- SparkContext can be accessed using `spark.sparkContext`

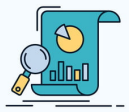


Scalable Data Processing

Resilient Distributed Datasets (RDDs)

PES UNIVERSITY

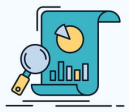
- RDD is the fundamental data abstraction in Spark
- It represents an immutable, distributed collection of objects
  - Resilient – fault tolerant
  - Distributed – spread across nodes
  - Dataset – collection of data elements



Scalable Data Processing

Key Properties of RDDs

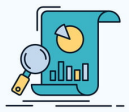
- RDDs have several defining characteristics
  - Immutable once created
  - Partitioned across cluster
  - Fault tolerant using lineage
- Transformations on RDDs always create new RDDs



Scalable Data Processing

RDD Creation Methods

- RDDs can be created in multiple ways
  - Parallelising existing collections
  - Loading data from external storage
- Data sources include HDFS, local filesystem, and object storage

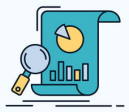


Scalable Data Processing

RDD Lineage and Fault Tolerance

PES UNIVERSITY

- RDDs track the sequence of operations that created them
- This sequence is known as lineage
  - Enables recomputation of lost partitions
  - Eliminates need for data replication
- Lineage is central to Spark's fault tolerance model

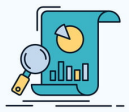


Scalable Data Processing

RDD Persistence and Caching

PES UNIVERSITY

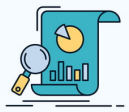
- RDDs can be cached in memory for reuse
- Caching improves performance for iterative workloads
  - `rdd.cache()`
  - `rdd.persist()`
- Cached data is reused across multiple actions



Scalable Data Processing

RDD Use Cases

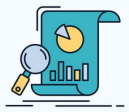
- RDDs are suitable for low-level data processing
- Preferred when fine-grained control is required
  - Unstructured data
  - Custom transformations
  - Legacy processing logic



Scalable Data Processing

Limitations of RDDs

- RDDs lack schema information
- Optimisation opportunities are limited
- Higher-level APIs are preferred for structured data

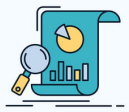


Scalable Data Processing

Summary – Spark and RDDs

PES UNIVERSITY

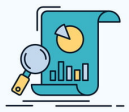
- Spark provides fast, scalable distributed computation
- Architecture separates coordination and execution
- RDDs form the foundation of Spark's data model



Scalable Data Processing

JOB Problem – Banana

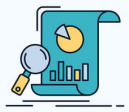
- Identify components of Spark architecture in a given diagram
- Explain the role of the driver program



Scalable Data Processing

JOB Problem – Orange

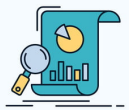
- Analyse why Spark performs better than MapReduce for iterative jobs
- Use RDD characteristics to justify your answer



Scalable Data Processing

JOB Problem – Jackfruit

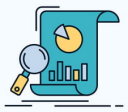
- Design a Spark-based architecture for log analysis
- Justify the use of RDDs over DataFrames



Scalable Data Processing

Brainstorming Questions

- Why is immutability important in distributed systems?
- How does lineage differ from replication?
- When should RDDs be avoided?



Scalable Data Processing

References

- Apache Spark Documentation v3.5.4
- PySpark User Guide v3.5.4
- Shilpi Saxena and Saurabh Gupta, *Practical Real-time Data Processing and Analytics*, Packt Publishing – Chapter 7



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 456



Scalable Data Processing

Batch Data Processing with PySpark

Niteesh K R

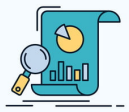
Computer Applications

Scalable Data Processing

Batch Data Processing with PySpark

RDDs – Transformations, Actions, KVPs

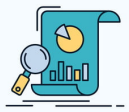
Niteesh K R
Computer Applications



Scalable Data Processing

RDD Transformations and Actions – Overview

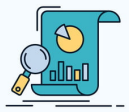
- RDD operations are classified into Transformations and Actions
- Transformations define logical data processing steps
- Actions trigger actual execution on the cluster
  - Transformations are lazy
  - Actions are eager



Scalable Data Processing

Working Dataset for RDD Examples

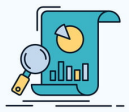
- Dataset stored in HDFS: `/data/sales.txt`
- Record format: `orderId,product,category,amount`
- Example:
  - `101,Phone,Electronics,45000`



Scalable Data Processing

RDD Transformation – map()

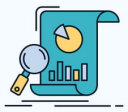
- Function: `map()`
- Parameters:
  - `func: Callable[[T], U]`
- Parameter Explanation:
  - `func` defines how each input element is converted into an output element
- Return Type:
  - `RDD[U]`
- Working:
  - Applies the function independently on each element
  - Output size remains same as input size
- Example:
  - `rdd.map(lambda x: x.split(","))`



Scalable Data Processing

RDD Transformation – filter()

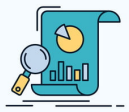
- Function: `filter()`
- Parameters:
  - `func: Callable[[T], bool]`
- Parameter Explanation:
  - `func` returns `True` for elements that should be retained
- Return Type:
  - `RDD[T]`
- Working:
  - Filters elements based on the given condition
  - Output RDD may have fewer elements
- Example:
  - `rdd.filter(lambda x: int(x[3]) > 30000)`



Scalable Data Processing

RDD Transformation – flatMap()

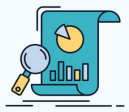
- Function: `flatMap()`
- Parameters:
  - `func: Callable[[T], Iterable[U]]`
- Parameter Explanation:
  - `func` maps each input element to zero or more output elements
- Return Type:
  - `RDD[U]`
- Working:
  - Similar to `map()` but flattens multiple outputs
  - Commonly used for tokenisation
- Example:
  - `rdd.flatMap(lambda x: x.split(","))`



Scalable Data Processing

RDD Transformation – distinct()

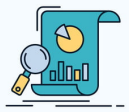
- Function: `distinct()`
- Parameters:
  - None
- Parameter Explanation:
  - Removes duplicate elements across all partitions
- Return Type:
  - `RDD[T]`
- Working:
  - Uses shuffle internally
  - May impact performance for large datasets
- Example:
  - `categories.distinct()`



Scalable Data Processing

RDD Action – collect()

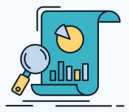
- Function: `collect()`
- Parameters:
  - None
- Parameter Explanation:
  - Returns all elements of the RDD to the driver program
- Return Type:
  - `List[T]`
- Working:
  - Triggers job execution
  - Suitable only for small datasets
- Example:
  - `rdd.collect()`



Scalable Data Processing

RDD Action – count()

- Function: `count()`
- Parameters:
  - None
- Parameter Explanation:
  - Computes the total number of elements in the RDD
- Return Type:
  - `int`
- Working:
  - Executes all pending transformations
- Example:
  - `rdd.count()`

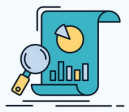


Scalable Data Processing

Key–Value Pair RDDs – Introduction

PES UNIVERSITY

- Key–Value Pair RDDs store data as `(key, value)`
- Keys enable grouping and aggregation operations

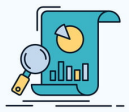


Scalable Data Processing

Creating Key–Value Pair RDDs

PES UNIVERSITY

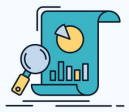
- KVP RDDs are created using `map()`
- Example:
  - `kv = rdd.map(lambda x:  (x[2], int(x[3])))`
- Category acts as key, amount acts as value



Scalable Data Processing

KVP Transformation – reduceByKey()

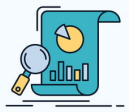
- Function: `reduceByKey()`
- Parameters:
  - `func: Callable[[V, V], V]`
- Parameter Explanation:
  - `func` defines how two values of the same key are merged
- Return Type:
  - `RDD[(K, V)]`
- Working:
  - Performs local aggregation before shuffle
  - Minimises network traffic
- Example:
  - `kv.reduceByKey(lambda a, b: a + b)`



Scalable Data Processing

KVP Transformation – groupByKey()

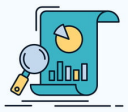
- Function: `groupByKey()`
- Parameters:
  - None
- Parameter Explanation:
  - Groups all values corresponding to the same key
- Return Type:
  - `RDD[(K, Iterable[V])]`
- Working:
  - Transfers all values across the network
  - Less efficient for large datasets
- Example:
  - `kv.groupByKey()`



Scalable Data Processing

reduceByKey vs groupByKey

- `reduceByKey` is more efficient than `groupByKey`
- Uses combiners to reduce shuffle size
- Preferred for aggregation operations

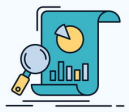


Scalable Data Processing

JOB Problems – Banana (Case-Based)

PES UNIVERSITY

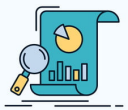
- Case 1: A text file contains one product name per line. You need to convert all product names to uppercase.
  - Identify the appropriate RDD transformation
- Case 2: A dataset contains sales records. You need to retain only records where the sale amount is greater than 10,000.
  - Choose the correct RDD operation and justify
- Case 3: A paragraph of text must be split into individual words for analysis.
  - Decide between `map()` and `flatMap()`



Scalable Data Processing

Banana Problems

- Case 4: You want to know how many records are present after filtering invalid entries.
  - Identify the suitable RDD action
- Case 5: A small RDD needs to be printed on the console for verification.
  - Which action should be used and why?

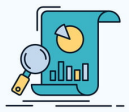


Scalable Data Processing

JOB Problems – Orange (Case-Based)

PES UNIVERSITY

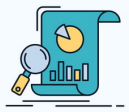
- Case 1: A dataset contains sales transactions across multiple categories. You need to calculate total sales per category.
  - Identify the sequence of RDD transformations required
- Case 2: A dataset contains repeated customer IDs. You are asked to find the unique customers.
  - Which RDD transformation should be applied?
- Case 3: An RDD is reused in multiple actions within a Spark application.
  - What optimisation technique should be applied and why?



Scalable Data Processing

Orange Problems

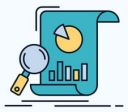
- Case 4: You need to group sales amounts for each product category before computing statistics.
  - Analyse whether `groupByKey()` is appropriate
- Case 5: A developer uses `collect()` on a large RDD and the job fails.
  - Explain the reason and suggest an alternative



Scalable Data Processing

JOB Problems – Jackfruit (Case-Based)

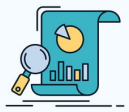
- Case 1: An e-commerce company wants to compute total revenue per category using Spark RDDs.
  - Design an efficient RDD pipeline
  - Justify each transformation used
- Case 2: A log analytics application groups events by user ID using `groupByKey()` and faces performance issues.
  - Analyse the root cause
  - Propose a better alternative
- Case 3: A Spark job performs multiple aggregations on the same dataset and runs slowly.
  - Suggest changes using RDD persistence



Scalable Data Processing

Jackfruit Problems

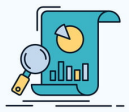
- Case 4: A dataset contains millions of key–value pairs with skewed keys.
  - Analyse how key skew affects `reduceByKey()`
  - Suggest mitigation strategies
- Case 5: A Spark application fails when a worker node crashes during execution.
  - Explain how RDD lineage enables recovery



Scalable Data Processing

Brainstorming Questions

- Why does Spark delay execution until an action is called?
- How does immutability help fault tolerance?



Scalable Data Processing

References

- PySpark API Reference v3.5.4
- PySpark RDD Programming Guide
- Shilpi Saxena and Saurabh Gupta, *Practical Real-time Data Processing and Analytics*, Packt Publishing – Chapter 8



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 456



Scalable Data Processing

Batch Data Processing with PySpark

Niteesh K R

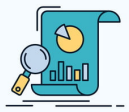
Computer Applications

Scalable Data Processing

Batch Data Processing with PySpark

EL II – PySpark Setup, RDDs, KVPs

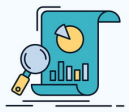
Niteesh K R
Computer Applications



Scalable Data Processing

Experiential Learning II – Objective

- Objective: To gain hands-on experience with PySpark setup and RDD-based processing
- This EL focuses on understanding Spark through practical scenarios
- Students will execute transformations, actions, and KVP operations
  - Environment setup
  - RDD operations
  - Key–Value Pair aggregations

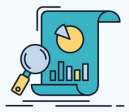


Scalable Data Processing

Case 1 – Setting Up PySpark Environment

PES UNIVERSITY

- Scenario: A data engineering intern must set up PySpark on a local machine
- Task:
  - Verify Spark installation
  - Launch PySpark shell
- Expected Setup Commands:
  - `export SPARK_HOME=/opt/spark`
  - `export PATH=$PATH:$SPARK_HOME/bin`
  - `pyspark`
- Outcome: SparkSession and SparkContext are initialised

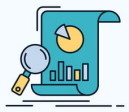


Scalable Data Processing

Case 2 – Verifying SparkContext

PES UNIVERSITY

- Scenario: The intern needs to confirm Spark is running correctly
- Task:
  - Access SparkContext from SparkSession
  - Check application name and master
- Example:
  - `sc = spark.sparkContext`
  - `sc.appName`
  - `sc.master`
- Outcome: Successful Spark cluster connection

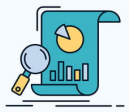


Scalable Data Processing

Case 3 – Creating RDD from Local Collection

PES UNIVERSITY

- Scenario: Sales amounts are available as a Python list
- Task: Create an RDD and perform basic inspection
- Example:
  - `sales = [12000, 45000, 8000, 30000]`
  - `rdd = sc.parallelize(sales)`
- Outcome: Distributed dataset created across partitions

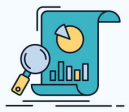


Scalable Data Processing

Case 4 – Applying RDD Transformations

PES UNIVERSITY

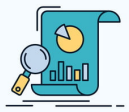
- Scenario: Only high-value sales should be analysed
- Task:
  - Filter sales above 20,000
  - Convert values to GST-inclusive amounts
- Example:
  - `filtered = rdd.filter(lambda x:  x > 20000)`
  - `gst = filtered.map(lambda x:  x * 1.18)`
- Outcome: Transformed RDD pipeline created



Scalable Data Processing

Case 5 – Triggering Execution Using Actions

- Scenario: The intern needs final results for reporting
- Task:
  - Count qualifying sales
  - Display final values
- Example:
  - `gst.count()`
  - `gst.collect()`
- Outcome: Spark executes the transformation pipeline

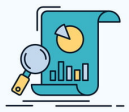


Scalable Data Processing

Case 6 – Creating RDD from File

PES UNIVERSITY

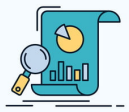
- Scenario: Sales records are stored in a CSV file
- File format: `orderId,product,category,amount`
- Task:
  - Load file into an RDD
  - Split records into fields
- Example:
  - `data = sc.textFile("/data/sales.txt")`
  - `records = data.map(lambda x:  x.split(","))`



Scalable Data Processing

Case 7 – Creating Key–Value Pair RDD

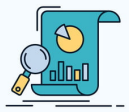
- Scenario: Total sales per category must be computed
- Task:
  - Use category as key
  - Use amount as value
- Example:
  - `kv = records.map(lambda x:  (x[2], int(x[3])))`
- Outcome: Key–Value Pair RDD created



Scalable Data Processing

Case 8 – Aggregation Using reduceByKey()

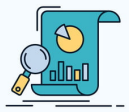
- Scenario: Aggregate total sales per category
- Task:
  - Apply reduceByKey
- Example:
  - `totals = kv.reduceByKey(lambda a, b:  a + b)`
- Outcome: Efficient aggregation with minimal shuffle



Scalable Data Processing

Case 9 – Analysing Output

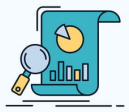
- Scenario: Results need to be verified
- Task:
  - Display aggregated results
  - Identify highest revenue category
- Example:
  - `totals.collect()`
- Outcome: Business insight derived from RDD processing



Scalable Data Processing

Experiential Learning II – Reflection

- Spark setup is lightweight and developer-friendly
- RDDs allow fine-grained control over data processing
- KVP RDDs enable efficient aggregation patterns
- Actions act as execution triggers



Scalable Data Processing

Experiential Learning II – Takeaways

- Spark separates logic definition from execution
- Transformations build lineage, actions trigger computation
- reduceByKey is preferred for aggregation



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 456



Scalable Data Processing

Batch Data Processing with PySpark

Niteesh K R

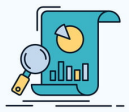
Computer Applications

Scalable Data Processing

Batch Data Processing with PySpark

Spark SQL – DataFrames, DDL, DML

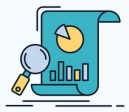
Niteesh K R
Computer Applications



Scalable Data Processing

Spark SQL – Motivation

- Spark SQL provides a structured data processing layer on top of Spark
- Enables working with structured and semi-structured data
- Combines:
  - SQL-style querying
  - DataFrame API (programmatic access)
- Designed to replace low-level RDD-based analytics

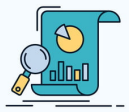


Scalable Data Processing

What is a DataFrame in Spark?

PES UNIVERSITY

- A DataFrame is a distributed collection of data organised into named columns
- Similar to:
  - Table in a relational database
  - DataFrame in Pandas (but distributed)
- Provides schema, optimisation, and declarative querying

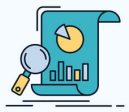


Scalable Data Processing

SparkSession and DataFrames

PES UNIVERSITY

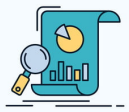
- SparkSession is the entry point to Spark SQL
- Responsible for:
  - Creating DataFrames
  - Executing SQL queries
- Example:
  - `from pyspark.sql import SparkSession`
  - `spark = SparkSession.builder.appName("SparkSQLDemo").getOrCreate()`



Scalable Data Processing

Creating DataFrames – From File

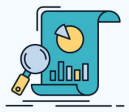
- DataFrames can be created from external data sources
- Common formats:
  - CSV
  - JSON
  - Parquet
- Example:
  - `df = spark.read.csv("/data/sales.csv", header=True, inferSchema=True)`



Scalable Data Processing

Understanding Schema

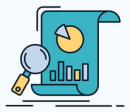
- Schema defines:
  - Column names
  - Data types
- Enables Spark to optimise query execution
- Example:
  - `df.printSchema()`



Scalable Data Processing

Basic DataFrame Operations

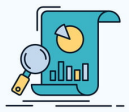
- Viewing data:
  - `df.show()`
  - `df.head()`
- Column selection:
  - `df.select("product", "amount")`
- Filtering rows:
  - `df.filter(df.amount > 20000)`



Scalable Data Processing

DataFrames vs SQL Tables

- DataFrames can be queried using SQL
- Requires registering a temporary view
- Example:
  - `df.createOrReplaceTempView("sales")`
- Enables seamless switching between API and SQL

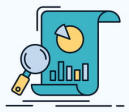


Scalable Data Processing

Spark SQL – DDL Operations

PES UNIVERSITY

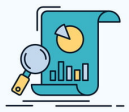
- DDL defines database structure
- Supported operations:
  - CREATE DATABASE
  - CREATE TABLE
  - DROP TABLE
- Executed using Spark SQL engine



Scalable Data Processing

CREATE DATABASE

- Used to logically organise tables
- Example:
  - `spark.sql("CREATE DATABASE IF NOT EXISTS retail")`
- Databases act as namespaces in Spark SQL

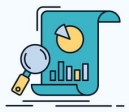


Scalable Data Processing

CREATE TABLE Using SQL

PES UNIVERSITY

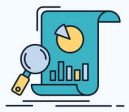
- Tables can be created using SQL syntax
- Example:
  - `spark.sql("CREATE TABLE sales (`
  - `product STRING, category STRING, amount INT)`
  - `USING parquet")`
- Schema is enforced at write time



Scalable Data Processing

DROP TABLE

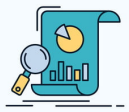
- Removes table metadata and data
- Example:
  - `spark.sql("DROP TABLE IF EXISTS sales")`
- Used during schema redesign or cleanup



Scalable Data Processing

Spark SQL – DML Operations

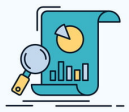
- DML manipulates table data
- Common operations:
  - INSERT
  - SELECT
  - FILTER



Scalable Data Processing

INSERT Data into Table

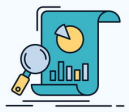
- Insert records using SQL
- Example:
  - `spark.sql("INSERT INTO sales VALUES`
  - `('Laptop','Electronics',45000)")`



Scalable Data Processing

SELECT Queries

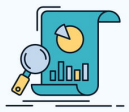
- Retrieve data using SQL
- Example:
  - `spark.sql("SELECT category, SUM(amount) FROM sales GROUP BY category")`
- Query optimisation handled by Catalyst Optimiser



Scalable Data Processing

DataFrame API vs SQL – Equivalence

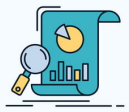
- DataFrame API:
  - `df.groupBy("category").sum("amount")`
- Equivalent SQL:
  - `SELECT category, SUM(amount) FROM sales GROUP BY category`
- Both generate the same execution plan



Scalable Data Processing

Why Prefer DataFrames Over RDDs?

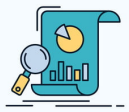
- Automatic optimisation via Catalyst
- Efficient execution using Tungsten
- Schema-aware processing
- Cleaner and more maintainable code



Scalable Data Processing

Summary

- DataFrames provide structured, optimised processing
- Spark SQL enables familiar relational operations
- DDL defines structure, DML manipulates data
- DataFrame API and SQL are interchangeable



Scalable Data Processing

References

- PySpark SQL API Documentation v3.5.4
- Spark SQL, DataFrames and Datasets Guide v3.5.4
- Shilpi Saxena and Saurabh Gupta, *Practical Real-time Data Processing and Analytics*, Packt Publishing – Chapter 6



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 456



Scalable Data Processing

Batch Data Processing with PySpark

Niteesh K R

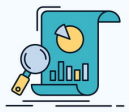
Computer Applications

Scalable Data Processing

Batch Data Processing with PySpark

DataFrames and SQL – Advanced Operations, UDFs

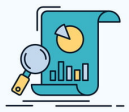
Niteesh K R
Computer Applications



Scalable Data Processing

Why “Advanced DataFrame + SQL” Matters

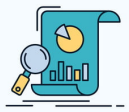
- In real Data Engineering work, most Spark pipelines are written using DataFrames and Spark SQL
  - Structured transformations are easier to maintain than RDD pipelines
  - Optimiser can generate better execution plans for DataFrames/SQL
- This topic focuses on “production-grade operations”:
  - Aggregations, filtering, sorting, grouping
  - Handling null values and dirty strings
  - Date/time parsing and transformations
  - UDFs (overview + safe usage)



Scalable Data Processing

Running Case Dataset – Retail Orders

- Scenario: A retail company stores orders coming from multiple channels
- Dataset columns (typical messy real-world data):
  - `order_id` (INT), `customer_id` (STRING), `city` (STRING)
  - `order_ts` (STRING), `amount` (DOUBLE), `discount` (DOUBLE)
  - `status` (STRING), `comments` (STRING)
- Typical issues:
  - `order_ts` is string (needs parsing)
  - `city/comments` contain extra spaces, mixed case, typos
  - `discount/comments` may be NULL



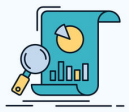
Scalable Data Processing

One Cohesive Example Program (Used Across Slides)

- We will keep one single program context and reuse it for all examples
- Setup (PySpark):

```
from pyspark.sql import functions as F
from pyspark.sql import types as T
df = spark.read.csv("/data/retail_orders.csv",
header=True, inferSchema=True)
df.createOrReplaceTempView("orders")
```

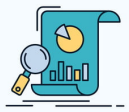
- When to use: Use this structure when you want both DataFrame API and SQL queries in the same notebook/script



Scalable Data Processing

Filtering – `filter()` / `where()`

- Function: `DataFrame.filter(condition)` (alias: `where(condition)`)
- Parameters (type):
  - `condition: Column or str`
- Parameter explanation:
  - `condition` specifies which rows should be retained (boolean expression)
- Return type:
  - `DataFrame`
- Working (case-based):
  - Case: Keep only “DELIVERED” orders above 10,000 amount for revenue analysis

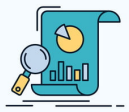


Scalable Data Processing

Example

- Example:

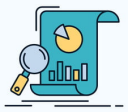
```
df2 = df.filter((F.col("status") == "DELIVERED") &
(F.col("amount") > 10000))
df3 = df.where("status = 'DELIVERED' AND amount >
10000")
```



Scalable Data Processing

Selecting Columns – `select()` and `selectExpr()`

- Function 1: `DataFrame.select(*cols)`
- Parameters (type):
  - `cols: Column or str (varargs)`
- Parameter explanation:
  - `cols` lists the columns/expressions you want in the output
- Return type: `DataFrame`
- Case: Build a clean reporting view with only required columns



Scalable Data Processing

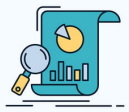
select()

- Example:

```
df.select("order_id","city","amount","status")
```

- Function 2: `DataFrame.selectExpr(*expr)`
- Parameters: `expr: str (varargs)` Parameter explanation: SQL-like expressions
- Return type: `DataFrame`
- Example:

```
df.selectExpr("order_id","upper(city) as
city_u","amount*0.1 as tax")
```

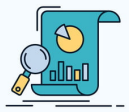


Scalable Data Processing

Creating/Transforming Columns – `withColumn()`

- Function: `DataFrame.withColumn(colName, col)`
- Parameters (type):
  - `colName: str`
  - `col: Column`
- Parameter explanation:
  - `colName` is the new/existing column name to write into
  - `col` is the expression used to compute the column value
- Return type: `DataFrame`
- Working (case-based):
  - Case: Create a `net_amount` column after applying discount (handling NULL discount safely later)
- Example:

```
df2 = df.withColumn("net_amount", F.col("amount")
- F.col("discount"))
```

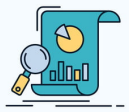


Scalable Data Processing

Sorting – `orderBy()` / `sort()`

- Function: `DataFrame.orderBy(*cols)` (alias: `sort(*cols)`)
- Parameters (type):
  - `cols: Column or str (varargs)`
- Parameter explanation:
  - Columns define ordering; can include `asc()/desc()` expressions
- Return type: `DataFrame`
- Case: Find top 10 highest value orders (descending amount)
- Example:

```
top10 =
df.orderBy(F.col("amount").desc()).limit(10)
```

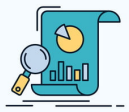


Scalable Data Processing

Limiting Rows – `limit()`

- Function: `DataFrame.limit(num)`
- Parameters (type):
  - `num: int`
- Parameter explanation:
  - `num` is the maximum number of rows to return
- Return type: `DataFrame`
- Working (case-based):
  - Case: Preview a small sample after cleaning transformations
- Example:

```
df.limit(5).show(truncate=False)
```

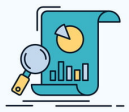


Scalable Data Processing

Grouping – `groupBy()`

- Function: `DataFrame.groupBy(*cols)`
- Parameters (type):
  - `cols: Column or str (varargs)`
- Parameter explanation:
  - `cols` specify grouping keys (e.g., city, status, customer_id)
- Return type:
  - `GroupedData`
- Working (case-based):
  - Case: Compute total net revenue per city
- Example:

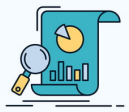
```
g = df.groupBy("city")
```



Scalable Data Processing

Aggregations – `agg()` on GroupedData

- Function: `GroupedData.agg(*exprs)` (or `agg(dict)`)
- Parameters (type):
  - `exprs: Column (varargs)` *or* `dict: Dict[str,str]`
- Parameter explanation:
  - `exprs` are aggregation expressions (sum/avg/count etc.)
  - `dict` maps column name to aggregation function name (simple usage)
- Return type: `DataFrame`

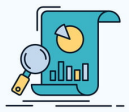


Scalable Data Processing

Example

- Case: Revenue by city with multiple metrics (count, sum, avg)
- Example:

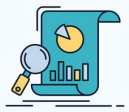
```
out = df.groupBy("city").agg(
  F.count("*").alias("orders"),
  F.sum("amount").alias("gross_revenue"),
  F.avg("amount").alias("avg_order_value"))
```



Scalable Data Processing

Common Aggregation Functions (Case-based)

- Function set (from `pyspark.sql.functions`):
  - `count(col)`, `sum(col)`, `avg(col)`, `min(col)`, `max(col)`
  - `countDistinct(*cols)`
- Parameters (type):
  - `col: Column or str` (for most)
  - `*cols: Column or str (varargs)` (for `countDistinct`)
- Parameter explanation:
  - Column(s) on which the aggregate is computed
- Return type:
  - `Column` (used inside `agg()` / `select()`)

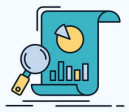


Scalable Data Processing

Aggregation Functions

- Case: For each `status`, compute volume and revenue summary
- Example:

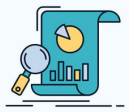
```
df.groupBy("status").agg(F.count("*").alias("cnt"),
F.sum("amount").alias("rev")).show()
df.select(F.countDistinct("customer_id")
.alias("unique_customers")).show()
```



Scalable Data Processing

DataFrame `join()` – Core for Real Pipelines

- Function: `DataFrame.join(other, on=None, how=None)`
- Parameters (type):
  - `other: DataFrame`
  - `on: str or List[str] or Column or List[Column] (optional)`
  - `how: str (optional)` (e.g., `"inner"`, `"left"`, `"right"`, `"full"`, `"left_semi"`)
- Parameter explanation:
  - `other` is the second table; `on` defines join condition; `how` defines join type
- Return type: `DataFrame`

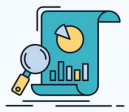


Scalable Data Processing

Join

- Case: Join orders with a `customers` DataFrame to enrich with customer tier
- Example:

```
cust = spark.read.csv("/data/customers.csv",
header=True, inferSchema=True)
enriched = df.join(cust, on="customer_id",
how="left")
```



Scalable Data Processing

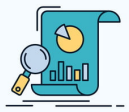
SQL Equivalence for Advanced Aggregation

- Case: “Revenue dashboard per city for delivered orders only”
- DataFrame API:

```
df.filter((F.col("status")=="DELIVERED"))
  .groupBy("city")
  .agg(F.count("*").alias("orders"),
F.sum("amount").alias("rev"))
  .orderBy(F.col("rev").desc())
```

- Equivalent SQL:

```
spark.sql("
SELECT city, COUNT(*) AS orders, SUM(amount) AS
rev
FROM orders WHERE status='DELIVERED'
GROUP BY city ORDER BY rev DESC")
```

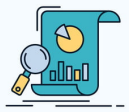


Scalable Data Processing

Null Handling – Why It Breaks Analytics

PES UNIVERSITY

- Null values commonly appear due to:
  - Missing discounts, missing comments, partial ingestion failures
  - Schema evolution where new columns arrive later
- Null-handling must be explicit to avoid:
  - Wrong revenue (NULL discount makes `amount - discount` become NULL)
  - Wrong filtering (NULL comparisons behave differently in SQL)

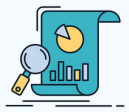


Scalable Data Processing

Null Handling – `isNull()` / `isNotNull()`

- Function: `Column.isNull()` and `Column.isNotNull()`
- Parameters:
  - None
- Parameter explanation:
  - Checks whether the column value is NULL for each row
- Return type:
  - `Column` (boolean expression)
- Case: Identify orders where discount is missing
- Example:

```
missing = df.filter(F.col("discount").isNull())
present = df.filter(F.col("discount").isNotNull())
```

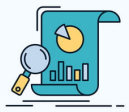


Scalable Data Processing

Null Handling – `coalesce()` for Safe Arithmetic

- Function: `F.coalesce(col1, col2, ...)`
- Parameters (type):
  - `colN: Column or str (varargs)`
- Parameter explanation:
  - Returns the first non-null value among the given columns/expressions
- Return type:
  - `Column`
- Case: Treat NULL discount as 0 to compute net amount correctly
- Example:

```
df2 = df.withColumn("discount_safe",
F.coalesce(F.col("discount"), F.lit(0.0)))
df3 = df2.withColumn("net_amount", F.col("amount")
- F.col("discount_safe"))
```

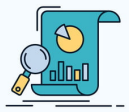


Scalable Data Processing

Null Handling – `na.fill()` and `na.drop()`

- Function 1: `DataFrame.na.fill(value, subset=None)`
- Parameters (type):
  - `value: int/float/str/bool or dict`
  - `subset: List[str] (optional)`
- Parameter explanation:
  - `value` is what replaces nulls; `subset` restricts which columns are affected
- Return type: `DataFrame`
- Case: Fill missing comments with “NA”, missing discount with 0
- Example:

```
df2 = df.na.fill("comments":"NA", "discount":0.0)
```

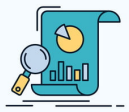


Scalable Data Processing

Null Handling

- Function 2: `DataFrame.na.drop(how="any", subset=None)`
- Parameters: `how: str, subset: List[str] (optional)`
  Parameter explanation: remove rows with nulls
- Return type: `DataFrame`
- Example:

```
df3 =
df.na.drop(subset=["order_id","amount","status"])
```

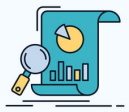


Scalable Data Processing

String Cleaning – `trim()`, `lower()`, `upper()`

- Function set: `F.trim(col)`, `F.lower(col)`, `F.upper(col)`
- Parameters:
  - `col: Column or str`
- Parameter explanation:
  - The input string column to be cleaned/transformed
- Return type: `Column`
- Case: City values contain extra spaces and mixed case (`" bengaluru ", "BENGALURU"`)
- Example:

```
df2 = df.withColumn("city_clean",
F.upper(F.trim(F.col("city"))))
```

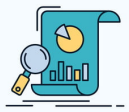


Scalable Data Processing

String Extraction – `substring()` and `split()`

- Function 1: `F.substring(col, pos, len)`
- Parameters:
  - `col:` `Column or str`
  - `pos:` `int` (1-based indexing)
  - `len:` `int`
- Parameter explanation:
  - Extracts `len` characters starting at position `pos`
- Return type: `Column`
- Case: Extract first 3 characters of city for quick grouping label
- Example:

```
df.withColumn("city_tag",
F.substring(F.col("city"), 1, 3))
```



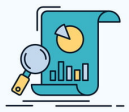
Scalable Data Processing

String Extraction

- Function 2: `F.split(str, pattern)`
- Parameters: `str: Column or str, pattern: str`

  Parameter explanation: splits string into array
- Return type: `Column` (array)
- Example:

```
df.withColumn("parts", F.split(F.col("comments"),
" "))
```

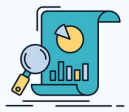


Scalable Data Processing

Date/Time – Parsing `order_ts` Safely

- Real pipelines often receive timestamps as strings
- Function: `F.to_timestamp(col, format=None)`
- Parameters:
  - `col: Column or str`
  - `format: str (optional)`
- Parameter explanation:
  - Converts string to timestamp; `format` defines parsing pattern when needed
- Return type: `Column` (timestamp)
- Case: `order_ts` is `"2026-02-04 10:15:00"`
- Example:

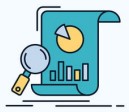
```
df2 = df.withColumn("order_time",
F.to_timestamp("order_ts", "yyyy-MM-dd HH:mm:ss"))
```



Scalable Data Processing

Date Functions – `year()`, `month()`, `datediff()`

- Function set:
  - `F.year(col)`, `F.month(col)`, `F.dayofmonth(col)`
  - `F.datediff(end, start)`
- Parameters:
  - For `year/month/dayofmonth`: `col: Column or str`
  - For `datediff`: `end: Column/str`, `start: Column/str`
- Parameter explanation:
  - Extract date parts; `datediff` returns days between two dates
- Return type: `Column`

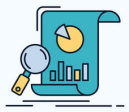


Scalable Data Processing

Example

- Case: Build monthly revenue and compute “days since order” for SLA analysis
- Example:

```
df3 = df2.withColumn("order_year",
F.year("order_time")).withColumn("order_month",
F.month("order_time"))
df4 = df3.withColumn("days_since_order",
F.datediff(F.current_date(),
F.to_date("order_time")))
```

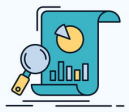


Scalable Data Processing

Formatting Dates – `date_format()` and `to_date()`

- Function 1: `F.to_date(col, format=None)`
- Parameters:
  - `col: Column or str`
  - `format: str (optional)`
- Parameter explanation: converts timestamp/string into date (drops time component)
- Return type: `Column` (date)
- Example:

```
df2 = df.withColumn("order_date",
F.to_date("order_ts", "yyyy-MM-dd HH:mm:ss"))
```



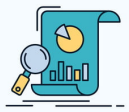
Scalable Data Processing

Date Handling

- Function 2: `F.date_format(dateExpr, format)`
- Parameters: `dateExpr: Column/str`, `format: str`

  Parameter explanation: formats date/timestamp as string
- Return type: `Column` (string)
- Example:

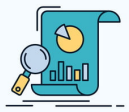
```
df2.withColumn("order_yyyymm",
F.date_format("order_time", "yyyy-MM")).show()
```



Scalable Data Processing

UDFs – Overview and When to Use

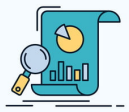
- UDF = User Defined Function (custom logic applied to columns)
- Use UDFs only when built-in functions cannot solve the problem cleanly
  - Built-in functions are faster and optimised
  - UDFs add overhead and can reduce optimisation opportunities
- Typical safe UDF use cases:
  - Complex business rule mapping not expressible via standard functions
  - Domain-specific parsing when regex/substring is insufficient



Scalable Data Processing

UDF Creation – `udf(func, returnType)`

- Function: `F.udf(f, returnType=None)`
- Parameters (type):
  - `f: Callable`
  - `returnType: DataType (optional)` (e.g., `T.StringType()`, `T.IntegerType()`)
- Parameter explanation:
  - `f` is your Python function; `returnType` tells Spark what type the UDF returns
- Return type:
  - `UserDefinedFunction`
- Working (case-based):
  - Case: Convert `status` into a business-friendly bucket for reporting



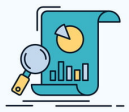
Scalable Data Processing

UDF Example

- Example:

```
def bucket(s):
  if s is None:  return "UNKNOWN"
  s = s.strip().upper()
  return "SUCCESS" if s in
["DELIVERED","COMPLETED"] else "NON_SUCCESS"
bucket_udf = F.udf(bucket, T.StringType())
df2 = df.withColumn("status_bucket",
bucket_udf(F.col("status")))
```

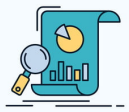
- When to use: Use UDF only when built-in `when/otherwise` becomes too complex or unreadable



Scalable Data Processing

Prefer Built-ins – `when()` and `otherwise()`

- Function: `F.when(condition, value).otherwise(value)`
- Parameters (type):
  - `condition:` `Column` (boolean)
  - `value:` `Any` (literal/column)
  - `otherwise_value:` `Any`
- Parameter explanation:
  - Creates SQL-like CASE WHEN logic without using a Python UDF
- Return type: `Column`
- Case: Flag high-value orders (`amount > 50000`) for priority processing



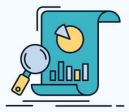
Scalable Data Processing

Usage of `when` and `otherwise`

- Example:

```
df2 = df.withColumn("priority",
F.when(F.col("amount") > 50000,
"HIGH").otherwise("NORMAL"))
```

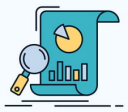
- Teaching note: This is typically faster than UDF and keeps the plan optimisable



Scalable Data Processing

Core Takeaways

- Advanced DataFrame operations are the foundation of Spark-based pipelines
  - `filter/where` for row selection
  - `groupBy + agg` for summarisation
  - `orderBy` for ranking and reporting
  - Built-in string/date/null functions to clean messy data
  - UDFs as a last resort for custom logic
- Strong DataFrame + SQL skills = faster development + exam confidence + industry relevance

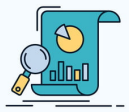


Scalable Data Processing

JOB Problems – Banana (Case-Based)

PES UNIVERSITY

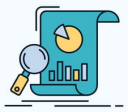
- Case 1: City values contain leading/trailing spaces and inconsistent case. Create a clean city column for grouping.
  - Suggest DataFrame functions to apply and the order of operations
- Case 2: Some orders have NULL discount. Compute `net_amount` without producing NULL results.
  - Identify the function(s) required to handle NULL safely
- Case 3: Retrieve only the orders whose status is DELIVERED and amount is above 20,000.
  - Write the operation using DataFrame API



Scalable Data Processing

Banana Problems

- Case 4: You need only `order_id, city, amount` columns for a report.
  - Identify the correct selection method(s)
- Case 5: Display top 5 highest-value orders.
  - Which function combination is appropriate?
- Case 6: Extract the year from `order_ts` for yearly trend analysis.
  - Name the function(s) used for parsing + year extraction

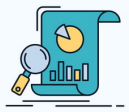


Scalable Data Processing

JOB Problems – Orange (Case-Based)

PES UNIVERSITY

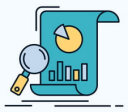
- Case 1: Compute revenue summary per city for DELIVERED orders: total revenue, average order value, total orders.
  - Provide the DataFrame pipeline using `groupBy` + `agg`
- Case 2: Identify “data quality risk”: rows missing `order_id` or `amount` should be removed.
  - Propose a null-handling approach using `na.drop` (subset-based)
- Case 3: Comments contain multiple exclamation marks and repeated spaces. Clean the comments column.
  - Specify an approach using `regexp_replace`



Scalable Data Processing

Orange Problems

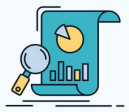
- Case 4: Build a monthly revenue report (YYYY-MM) and sort months in increasing order.
  - Specify the date parsing + formatting functions + ordering step
- Case 5: Enrich orders with customer tier using a left join on `customer_id`.
  - Specify join type and the function call with parameters
- Case 6: Write the “revenue by city” pipeline in both DataFrame API and Spark SQL.
  - Mention why both are equivalent for execution planning



Scalable Data Processing

JOB Problems – Jackfruit (Case-Based)

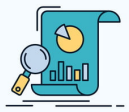
- Case 1: Build a “clean + aggregate + rank” pipeline for a dashboard:
  - Clean city (`trim + upper`), fill missing discount with 0, compute net amount
  - Group by city, compute revenue, and return top 10 cities
- Case 2: A team used a Python UDF to clean city names. The pipeline became slow.
  - Analyse why UDF may reduce performance
  - Replace the UDF with equivalent built-in functions
- Case 3: Orders arrive with multiple timestamp formats from different sources.
  - Propose a robust parsing strategy and explain failure scenarios (NULL after parsing)



Scalable Data Processing

Jackfruit Problems

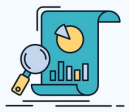
- Case 4: Data skew: one city has 60% of the records, causing aggregation tasks to be slow.
  - Explain why skew impacts `groupBy + agg`
  - Suggest mitigation ideas (conceptual)
- Case 5: A compliance report requires:
  - Drop rows with missing IDs, standardise city, remove symbols from comments
  - Generate: total orders, unique customers, and monthly revenue
- Explain a complete plan and justify each operation category (null/string/date/agg)



Scalable Data Processing

Brainstorming Questions

- Why does Spark encourage DataFrames/SQL over RDDs for analytics?
- When does NULL become “dangerous” in arithmetic and filtering?
- Why is `coalesce()` preferred for safe defaulting in computations?
- How do string cleaning errors propagate into wrong grouping results?
- Why should UDFs be treated as a last resort in Spark SQL pipelines?
- If both SQL and DataFrame API are equivalent, why might a team still prefer one over the other?



Scalable Data Processing

References

- PySpark SQL API Reference v3.5.4
- PySpark SQL Functions (`pyspark.sql.functions`) v3.5.4
- Spark SQL, DataFrames and Datasets Guide v3.5.4
- PySpark User Guide – (Optional) Pandas API on Spark v3.5.4 (UDF alternatives and expressiveness)
- Shilpi Saxena and Saurabh Gupta, *Practical Real-Time Data Processing and Analytics*, Packt Publishing (2017) – Chapter 9 (Working with Spark) and Chapter 10 (Working with Spark Operations)



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 456



Scalable Data Processing

Batch Data Processing with PySpark

Niteesh K R

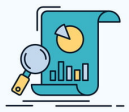
Computer Applications

Scalable Data Processing

Batch Data Processing with PySpark

EL III – DataFrames and Spark SQL, Partitioned Tables

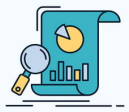
Niteesh K R
Computer Applications



Scalable Data Processing

Experiential Learning III – Objective

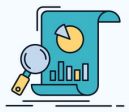
- This EL focuses on integrating DataFrames API and Spark SQL in a single workflow
- Students will practice:
  - Reading structured data (CSV and JSON)
  - Performing DDL and DML using DataFrames and Spark SQL
  - Aggregations, filtering, grouping, and sorting
  - Creating and querying partitioned tables
- Emphasis is on:
  - Correct usage patterns
  - Observing execution behaviour
  - Understanding when to use DataFrame API vs SQL



Scalable Data Processing

Scenario and Dataset

- Scenario: An e-commerce analytics team processes daily order data
- Data arrives in two formats:
  - CSV files from batch ingestion
  - JSON files from upstream services
- Typical attributes:
  - `order_id`, `customer_id`, `city`
  - `order_date`, `amount`, `status`
- Data will be used to create analytical tables and reports



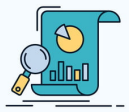
Scalable Data Processing

Step 1 – Reading CSV and JSON Data

- Load structured data using DataFrame readers
- Students should verify:
  - Schema inference
  - Correct data types
  - Handling of malformed rows
- Reference commands:

```
spark.read.csv(path, header=True,
inferSchema=True)
spark.read.json(path)
```

- Teaching note:
  - Highlight differences in schema handling between CSV and JSON



Scalable Data Processing

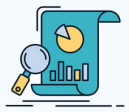
Step 2 – Creating Views for Spark SQL

PES UNIVERSITY

- Register DataFrames as temporary views
- Enables switching between:
  - DataFrame API
  - Spark SQL queries
- Reference command:

```
df.createOrReplaceTempView("orders")
```

- Teaching focus:
  - SQL operates on logical plans, not raw files
  - Views are session-scoped



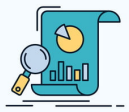
Scalable Data Processing

Step 3 – DDL Operations Using Spark SQL

- Perform schema-level operations using SQL
- Typical DDL tasks:
  - Creating databases
  - Creating managed tables
  - Defining column types
- Reference SQL:

```
CREATE DATABASE IF NOT EXISTS sales_db
CREATE TABLE orders_tbl (...)
```

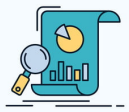
- Teaching note:
  - Distinguish between temporary views and persistent tables



Scalable Data Processing

Step 4 – DML Operations Using DataFrames and SQL

- Perform data manipulation:
  - Filtering records
  - Selecting required columns
  - Sorting and limiting results
- Students should practice:
  - Same operation via DataFrame API
  - Equivalent operation via SQL
- Teaching focus:
  - Emphasise logical equivalence, not syntax memorisation

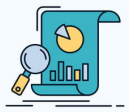


Scalable Data Processing

Step 5 – Aggregations and Grouping

PES UNIVERSITY

- Common analytical questions:
  - Total revenue per city
  - Number of orders per day
  - Average order value per status
- Use:
  - `groupBy + agg` (DataFrame API)
  - `GROUP BY` (Spark SQL)
- Teaching note:
  - Reinforce that aggregation triggers shuffles



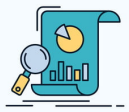
Scalable Data Processing

Step 6 – Creating Partitioned Tables

- Partitioning improves query performance by reducing data scanned
- Typical partition columns:
  - Date (`order_date`)
  - Region or city
- Reference SQL:

```
CREATE TABLE orders_part (...)  PARTITIONED BY
(order_date)
```

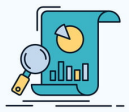
- Teaching focus:
  - Partition column should be frequently used in filters



Scalable Data Processing

Step 7 – Writing Data into Partitioned Tables

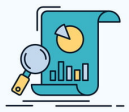
- Insert data using:
  - DataFrame write API
  - Spark SQL `INSERT INTO`
- Students should observe:
  - Directory structure created in storage
  - One folder per partition value
- Teaching note:
  - Explain how partitions map to filesystem paths



Scalable Data Processing

Step 8 – Querying Partitioned Tables

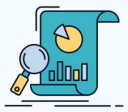
- Execute queries with and without partition filters
- Observe:
  - Faster queries when partition column is used in WHERE clause
  - Reduced data scanning
- Teaching focus:
  - Partition pruning concept
  - Why wrong partition choice degrades performance



Scalable Data Processing

Key Observations and Learning Outcomes

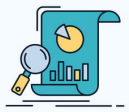
- DataFrames and Spark SQL are complementary, not competing tools
- Partitioning is a physical optimisation, not a logical one
- Correct table design directly impacts performance
- Most real-world Spark jobs combine:
  - File ingestion
  - Data cleaning
  - SQL-based analytics



Scalable Data Processing

Brainstorming Questions

- When would you prefer DataFrame API over Spark SQL?
- Why is partitioning more effective for filtering than sorting?
- What happens if a partition column has too many distinct values?
- Why are partition columns usually excluded from aggregations?
- How does partitioning differ from bucketing conceptually?



Scalable Data Processing

References

- Spark SQL, DataFrames and Datasets Guide v3.5.4
- PySpark SQL API Reference v3.5.4
- Spark SQL – DDL Statements
- Shilpi Saxena and Saurabh Gupta, *Practical Real-Time Data Processing and Analytics*, Packt Publishing – Chapter 9 (Spark SQL and DataFrames)



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 456



Scalable Data Processing

Batch Data Processing with PySpark

Niteesh K R

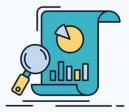
Computer Applications

Scalable Data Processing

Batch Data Processing with PySpark

JOB Problems – II

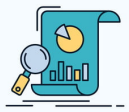
Niteesh K R
Computer Applications



Scalable Data Processing

JOB Problems – Banana (Easy) – Part 1

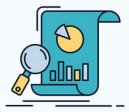
- Case 1: Read a CSV file containing student marks using PySpark and display the schema and first 10 rows.
- Case 2: Create an RDD from a text file and count the total number of lines.
- Case 3: Given an RDD of integers, filter out odd numbers and collect the result.
- Case 4: Load a CSV file and select only required columns using DataFrame API.
- Case 5: Create an RDD from a Python list and compute the sum using an action.
- Case 6: Convert all strings in an RDD to uppercase.
- Case 7: Load a JSON file and verify inferred schema correctness.



Scalable Data Processing

JOB Problems – Banana (Easy) – Part 2

- Case 8: Filter DataFrame rows where `status = 'ACTIVE'`.
- Case 9: Create a Key-Value RDD of (`product`, `price`) and display it.
- Case 10: Count the number of distinct cities in a DataFrame.
- Case 11: Sort a DataFrame based on a numeric column in descending order.
- Case 12: Display top 5 highest values from a DataFrame column.
- Case 13: Convert a DataFrame into an RDD and retrieve the first record.
- Case 14: Create a temporary view and execute a basic Spark SQL SELECT query.



Scalable Data Processing

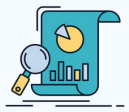
JOB Problems – Orange (Medium) – Part 1

- Case 1: Read a CSV file of sales data and compute total revenue using DataFrame aggregations.
- Case 2: Implement Word Count using RDD transformations and actions.
- Case 3: Group orders by city and compute order count per city.
- Case 4: Create a KVP RDD of (`customer_id`, `order_amount`) and compute total spend per customer.
- Case 5: Load two CSV files and perform an inner join using DataFrame API.
- Case 6: Filter records belonging to a specific year using Spark date functions.

JOB Problems – Orange (Medium) – Part 2

Case 7: Replace NULL values in numeric columns and compute derived columns safely.

- Case 8: Sort records by multiple columns (`date`, `amount`).
- Case 9: Using Spark SQL, compute average salary department-wise.
- Case 10: Create a partitioned table based on date and insert data into it.
- Case 11: Query partitioned tables using partition filters and observe behaviour.
- Case 12: Clean string columns using trim and case conversion functions.
- Case 13: Convert timestamp strings into date and timestamp columns.
- Case 14: Write a DataFrame pipeline and its equivalent Spark SQL query.



Scalable Data Processing

JOB Problems – Jackfruit (Difficult) – Part 1

- Case 1: Build a full PySpark ETL pipeline to read CSV orders, clean data, aggregate revenue per city, and store results.
- Case 2: Analyse a skewed dataset where one city dominates records and explain the impact on aggregation.
- Case 3: Implement customer spend analysis using KVP RDDs and compare with DataFrame implementation.
- Case 4: Read nested JSON data, flatten required fields, and store results as a managed table.
- Case 5: Design a pipeline to safely handle missing dates, malformed strings, and NULL numeric fields.

JOB Problems – Jackfruit (Difficult) – Part 2

- Case 6: Generate monthly revenue trends using Spark SQL date functions.
- Case 7: Write a UDF to categorise customers based on spending behaviour.
- Case 8: Replace a UDF-based solution with built-in functions and justify performance benefits.
- Case 9: Demonstrate partition pruning by querying partitioned data with and without filters.
- Case 10: Build an end-to-end job combining RDDs, DataFrames, and Spark SQL.
- Case 11: Analyse an aggregation job conceptually to identify shuffle stages.
- Case 12: Design a reporting pipeline to compute top-N results per category.



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 456